Topics in Software Engineering – Software Reengineering

By

Dr. Junaid Akram

Assistant Professor, Department of Computer Science COMSATS (Lahore)

Software Evolution

Concepts and Preliminaries

Software Evolution

- In 1965, Mark Halpern used the term *evolution* to define the dynamic growth of software.
- The term evolution in relation to application systems took gradually in the 1970s.
- Lehman and his collaborators from IBM are generally credited with pioneering the research field of software evolution.
- Lehman formulated a set of observations that he called laws of evolution.
- These laws are the results of studies of the evolution of large-scale proprietary or closed source system (CSS).
- The laws concern what Lehman called E-type systems:
  - “Monolithic systems produced by a team within an organization that solve a real-world problem and have human users.”

Software Evolution: Laws of Lehman

Continuing change (1st) – A system will become progressively less satisfying to its user over time, unless it is continually adapted to meet new needs.

Increasing complexity (2nd) – A system will become progressively more complex, unless work is done to explicitly reduce the complexity.

Self-regulation (3rd) – The process of software evolution is self regulating with respect to the distributions of the products and process artifacts that are produced.

Conservation of organizational stability (4th) – The average effective global activity rate on an evolving system does not change over time, that is the average amount of work that goes into each release is about the same.

Software Evolution: Laws of Lehman

Conservation of familiarity (5th) – The amount of new content in each successive release of a system tends to stay constant or decrease over time.

Continuing growth (6th) – The amount of functionality in a system will increase over time, in order to please its users.

Declining quality (7th) – A system will be perceived as losing quality over time, unless its design is carefully maintained and adapted to new operational constraints.

Feedback system (8th) – Successfully evolving a software system requires recognition that the development process is a multi-loop, multi-agent, multi-level feedback system.

Software Change

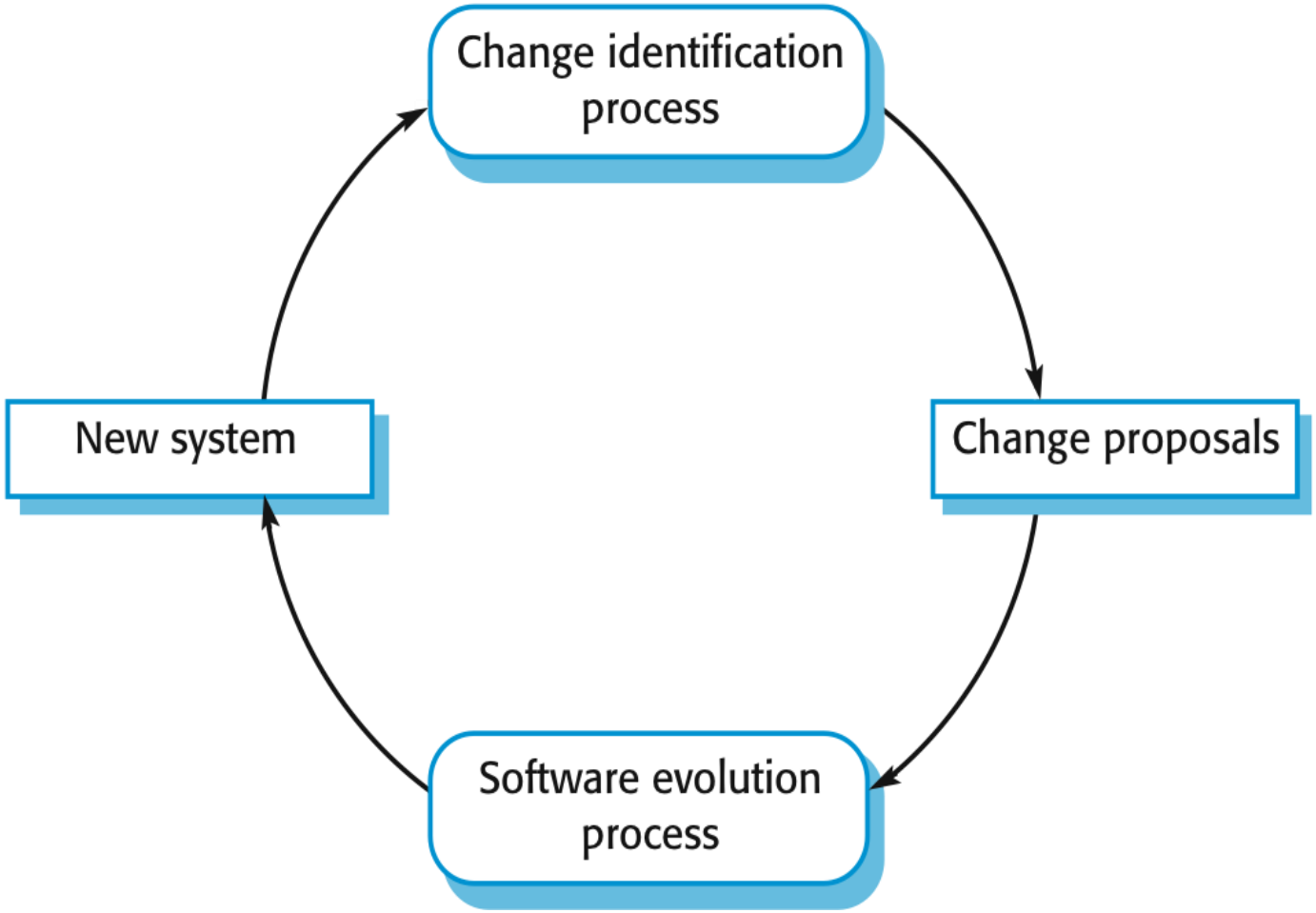
- Software change is inevitable:
  - New requirements emerge when the software is used
  - The business environment changes
  - Errors must be repaired
  - New computers and equipment are added to the system
  - The performance or reliability of the system may have to be improved

- A key problem for all organizations is implementing and managing change to their existing software systems

Software Change

- Software development does not stop when a system is delivered but continues throughout the lifetime of the system.
- After a system has been deployed, it inevitably has to change if it is to remain useful.
- Business changes and changes to user expectations generate new requirements for the existing software.
- Parts of the software may have to be modified to correct errors that are found in operation,
  - to adapt it for changes to its hardware and software platform,
  - to improve its performance or other non-functional characteristics.

Change identification and Evolution Processes



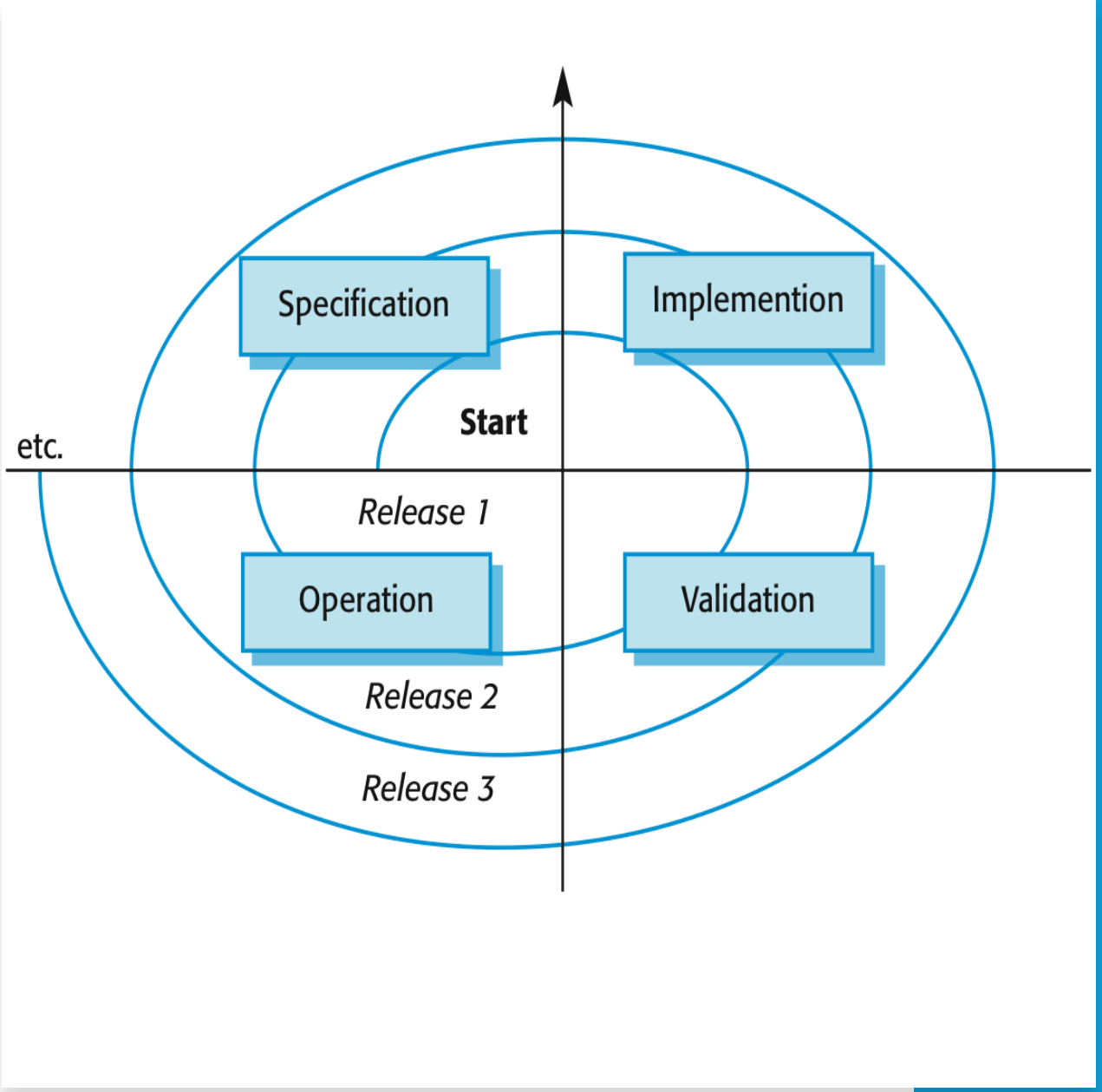
Importance of Evolution

- Software evolution is important because organizations have invested large amounts of money in their software and are now completely dependent on these systems.
- Their systems are critical business assets and they have to invest in system change to maintain the value of these assets.
- Consequently, most large companies spend more on maintaining existing systems than on new systems development.
- Therefore, new releases of the systems, incorporating changes, and updates, are usually created at regular intervals.

Importance of Evolution

- Based on an informal industry poll, Erlikh (2000) suggests that 85–90% of organizational software costs are evolution costs.
- Other surveys suggest that about two-thirds of software costs are evolution costs.
- For sure, the costs of software change are a large part of the IT budget for all companies.
- Software cost a lot of money so a company has to use a software system for many years to get a return on its investment.

Spiral Model of Development and Evolution



Spiral Model of Development and Evolution

- This model of software evolution implies that a single organization is responsible for both the initial software development and the evolution of the software.
- Most packaged software products are developed using this approach.
- For custom software, a different approach is commonly used.
- A software company develops software for a customer and the customer's take over the system. They are responsible for software evolution.

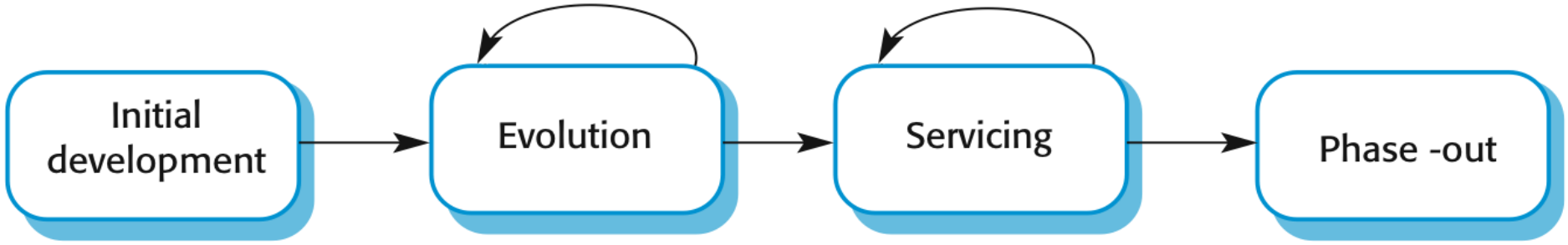
Spiral Model of Development and Evolution

- Alternatively, the software customer might issue a separate contract to a different company for system support and evolution.
- In this case, there are likely to be discontinuities in the spiral process.
- Requirements and design documents may not be passed from one company to another.
- Companies may merge or reorganize and inherit software from other companies, and then find that this has to be changed.
- Maintenance involves extra process activities, such as program understanding, in addition to the normal activities of software development.

Alternative View

- Rajlich and Bennett (2000) proposed an alternative view of the software evolution life cycle.
- In this model, they distinguish between Evolution and Servicing.
  - Evolution is the phase in which significant changes to the software architecture and functionality may be made.
  - During servicing, the only changes that are made are relatively small, essential changes.

Evolution and Servicing



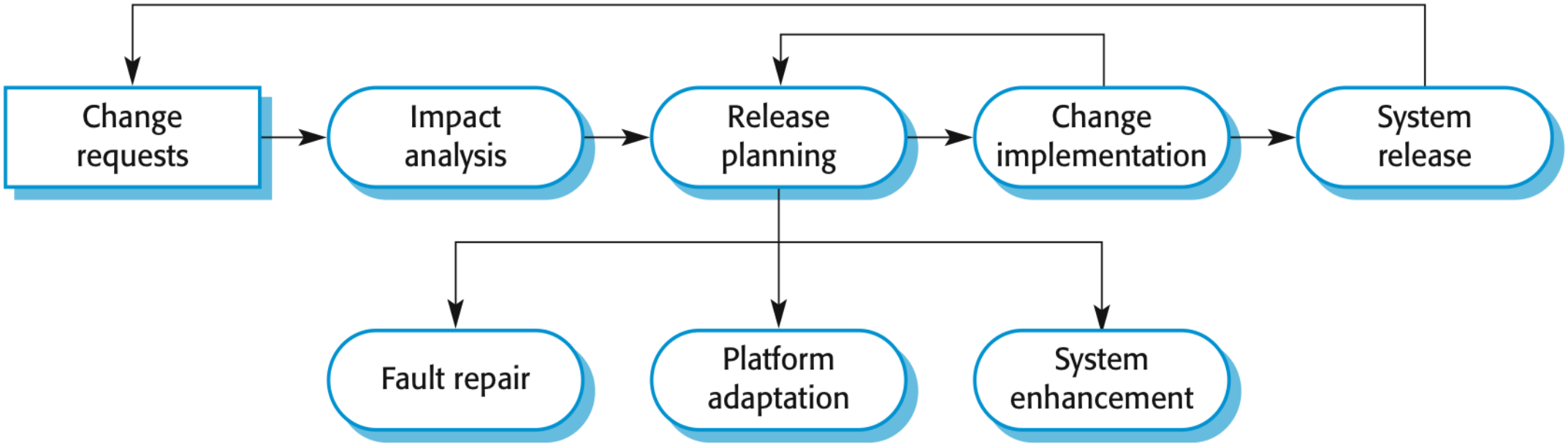
Evolution and Servicing

- Evolution
  - The stage in a software system's life cycle where it is in operational use and is evolving as new requirements are proposed and implemented in the system
- Servicing
  - At this stage, the software remains useful but the only changes made are those required to keep it operational, i.e. bug fixes and changes to reflect changes in the software's environment. No new functionality is added
- Phase-out
  - The software may still be used but no further changes are made to it

Evolution Processes

- Software evolution processes depend on
  - The type of software being maintained
  - The development processes used
  - The skills and experience of the people involved
- Proposals for change are the driver for system evolution
  - Should be linked with components that are affected by the change, thus allowing the cost and impact of the change to be estimated
- Change identification and evolution continues throughout the system lifetime

Software Evolution Process Model



Evolution Process Activities

- The cost and impact of these changes are assessed to see how much of the system is affected by the change and how much it cost to implement the change.
- If the proposed changes are accepted, a new release of the system is planned.
- During release planning, all proposed changes (fault repair, adaptation, and new functionality) are considered.
- A decision is then made on which changes to implement in the next version of the system.
- The changes are implemented and validated, and a new version of the system is released.
- The process then iterates with a new set of changes proposed for the next release.

Architectural Evolution

- There is a need to convert many legacy systems from a centralised architecture to a client-server architecture
- Change drivers
  - Hardware costs. Servers are cheaper than mainframes
  - User interface expectations. Users expect graphical user interfaces
  - Distributed access to systems. Users wish to access the system from different, geographically separated, computers

Evolution Change implementation

- Iteration of the development process where the revisions to the system are designed, implemented and tested
- A critical difference is that the first stage of change implementation may involve program understanding, especially if the original system developers are not responsible for the change implementation
- During the program understanding phase, you have to understand how the program is structured, how it delivers functionality and how the proposed change might affect the program

Cont..

- Ideally, the change implementation stage of this process should modify the system specification, design, and implementation to reflect the changes to the system
- New requirements that reflect the system changes are proposed, analyzed, and validated.
- System components are redesigned and implemented and the system is retested. If appropriate, prototyping of the proposed changes may be carried out as part of the change analysis process.

Urgent Change Requests

- Urgent changes may have to be implemented without going through all stages of the software engineering process
  - If a serious system fault has to be repaired to allow normal operation to continue
  - If changes to the system's environment (e.g., an OS upgrade) have unexpected effects
  - If there are business changes that require a very rapid response (e.g. the release of a competing product)

The Emergency Repair Process

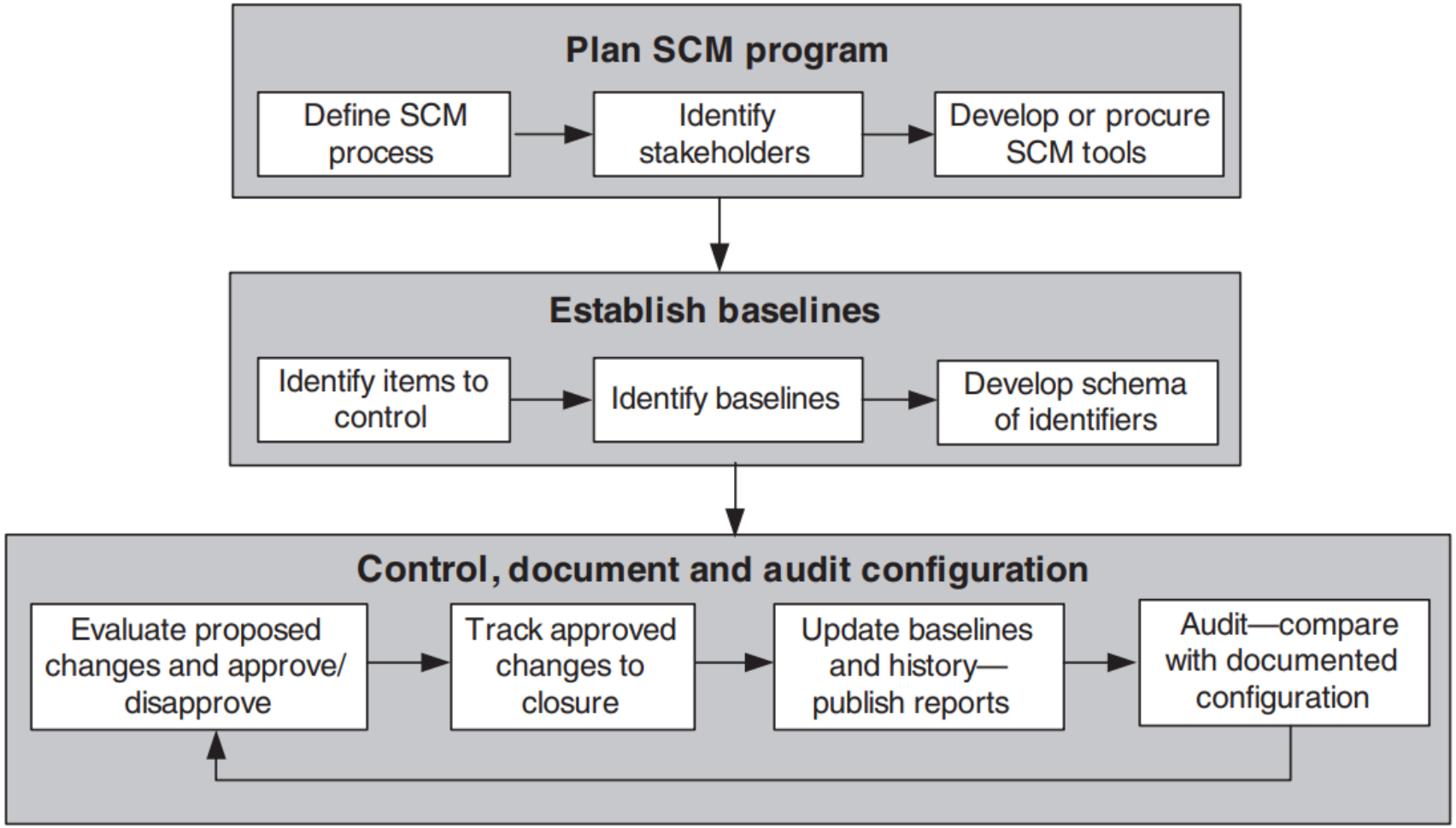
- In these cases, the need to make the change quickly means that you may not be able to follow the formal change analysis process.
- Rather than modify the requirements and design, you make an emergency fix to the program to solve the immediate problem.
- However, the danger is that the requirements, the software design, and the code become inconsistent.
- Although you may intend to document the change in the requirements and design, additional emergency fixes to the software may then be needed.

The Emergency Repair Process

- These take priority over documentation. Eventually, the original change is forgotten and the system documentation and code are never realigned.
- Emergency system repairs usually have to be completed as quickly as possible. You chose a quick and workable solution rather than the best solution as far as system structure is concerned.
- This accelerates the process of software ageing so that future changes become progressively more difficult and maintenance costs increase.

Software Configuration Management (Evolution)

implementation



Case Study

Evolution in Opensource Software

Free and Open Source Software (FOSS)!

- Over the past 2–3 decades, there has been a massive shift in the way software is being reengineered and maintained
- The major players responsible for this paradigm shift is open source.
- Generally, open source refers to a computer program in which the source code is available to the general public for use or modification from its original design.
- Open-source code is meant to be a collaborative effort where programmers improve upon the source code and share the changes within the community.
- This paradigm shift has led to an increase in technological advancement
- **Linux, LibreOffice, MySQL, Firefox, GIMP,** and **Blender** are some examples of free and open-source software.

Linux Kernel Evolution: Case study 1

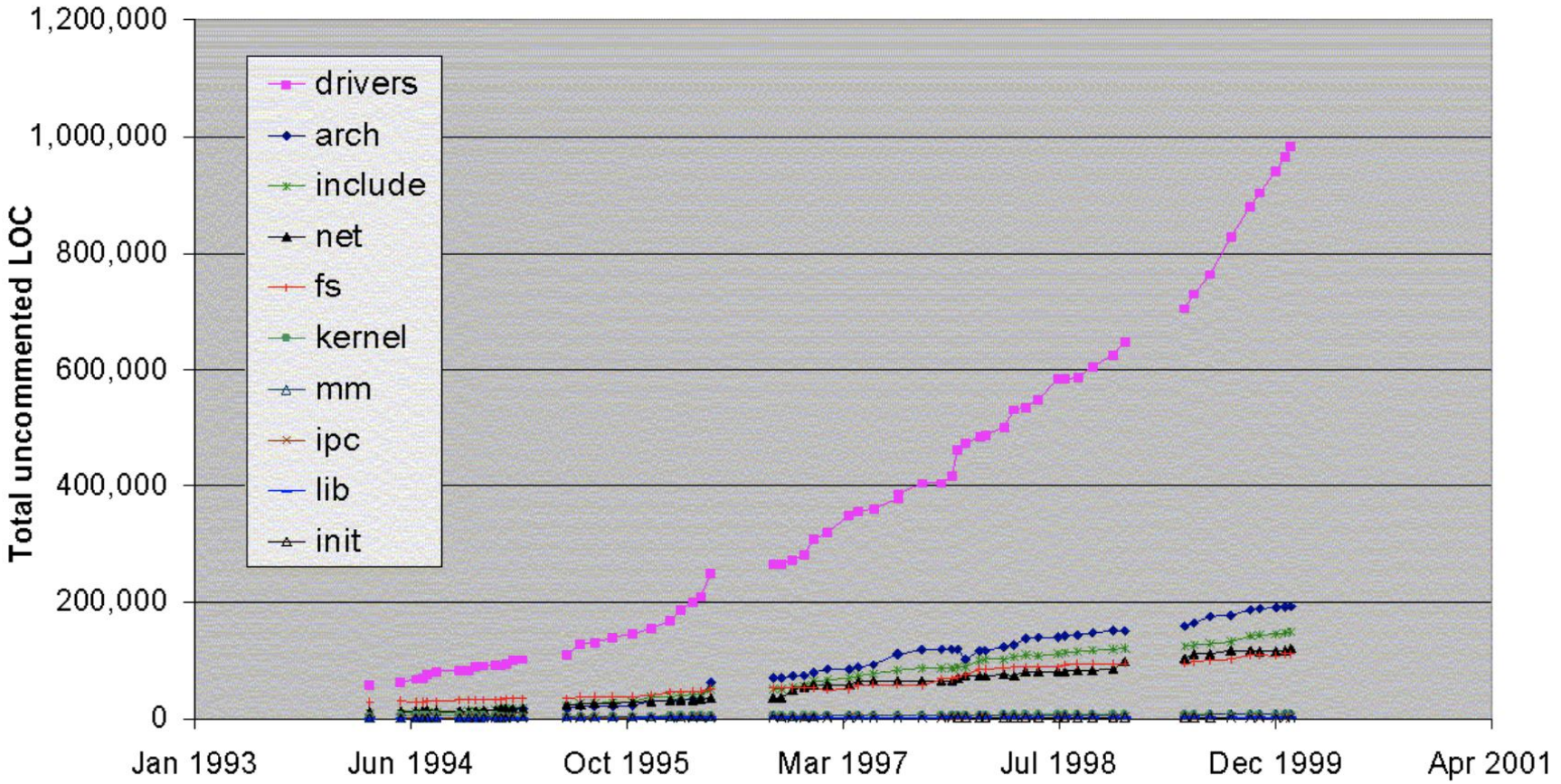
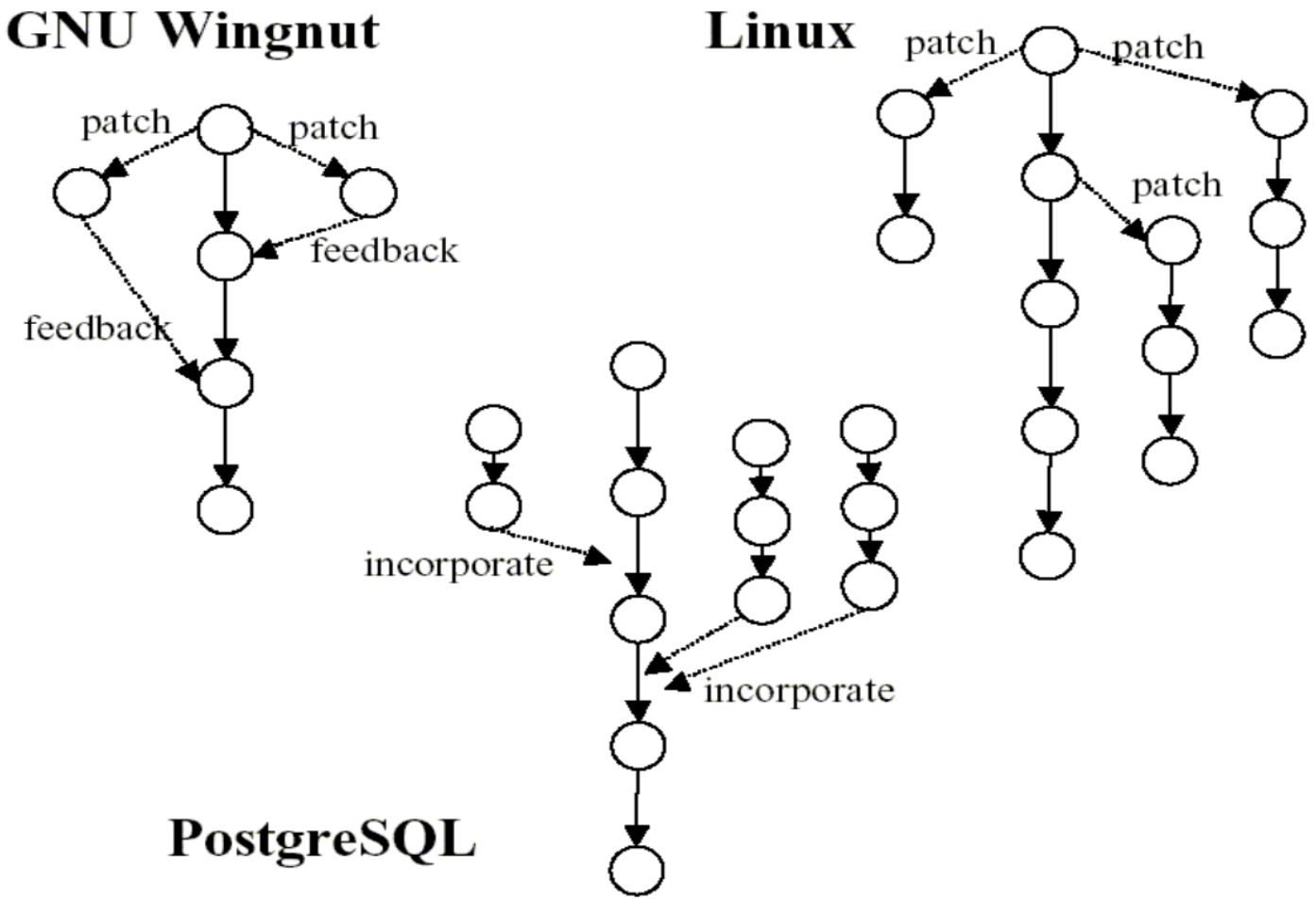


Figure 1. Data revealing the size and growth of major sub-systems in the Linux Kernel during 1994-1999 [Source: Godfrey and Tu 2000].

Source: https://ics.uci.edu/~wscacchi/Papers/New/Understanding-OSS-Evolution.pdf

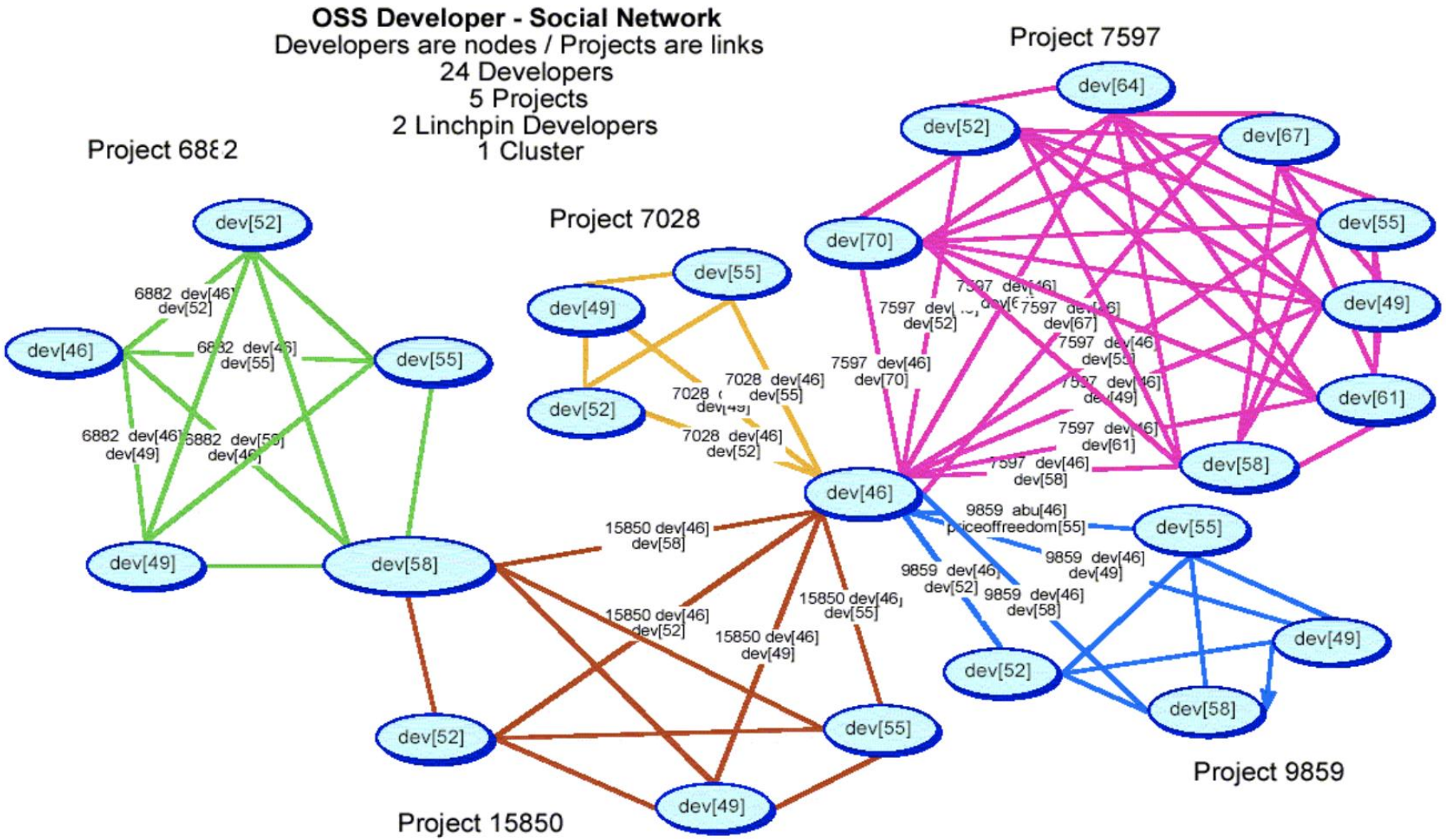
Patterns of Software System Evolution



Patterns of software system evolution forking and joining across releases (nodes in each graph) for four different F/OSS systems [Source: Nakakoji, Yamamoto, et al., 2002]

Source: https://ics.uci.edu/~wscacchi/Papers/New/Understanding-OSS-Evolution.pdf

Social Network of FOSS developers



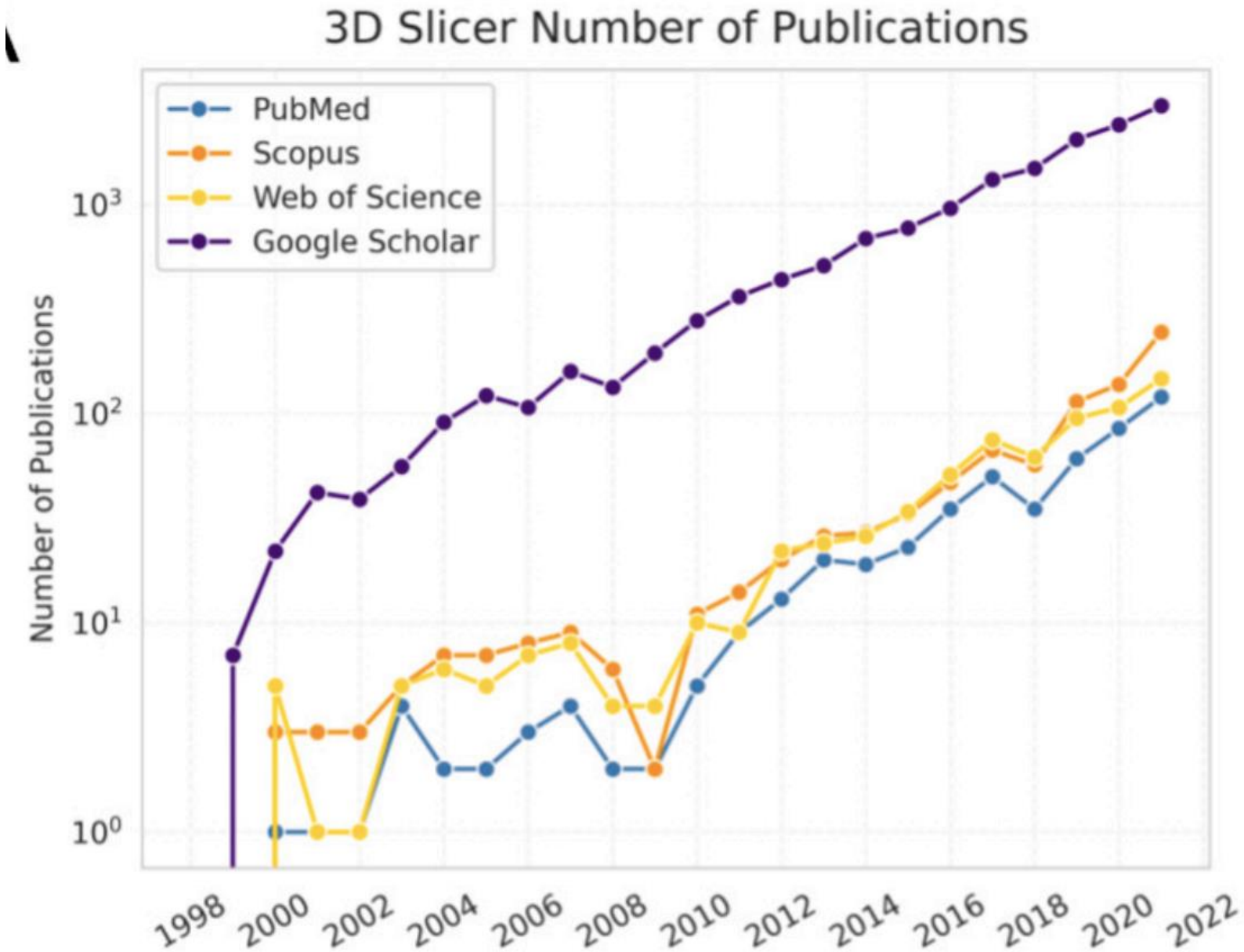
A social network of F/OSS developers that interlinks five different projects through two linchpin developers, dev[46] and dev[58] [Source: Madey, Freeh, and Tynan 2002].

Source: https://ics.uci.edu/~wscacchi/Papers/New/Understanding-OSS-Evolution.pdf

A case study of medical image software evolution: Case study 2 :3D Slicer

- 3D Slicer was born as a master's thesis project between the Surgical Planning Laboratory at and the Massachusetts Institute of Technology (MIT) Artificial Intelligence Laboratory, US, in 1998.
- 3D Slicer source code is released under the "3D Slicer Software License", a open-source license
  - Slicer1 and Slicer2 source codes were located in a Concurrent Versions System (CVS) repository.
  - Slicer3 source codes were located in a Subversion (SVN) repository1.
  - Slicer4 source codes are located in a web2, while the developing version can be found in its official GitHub repository3.

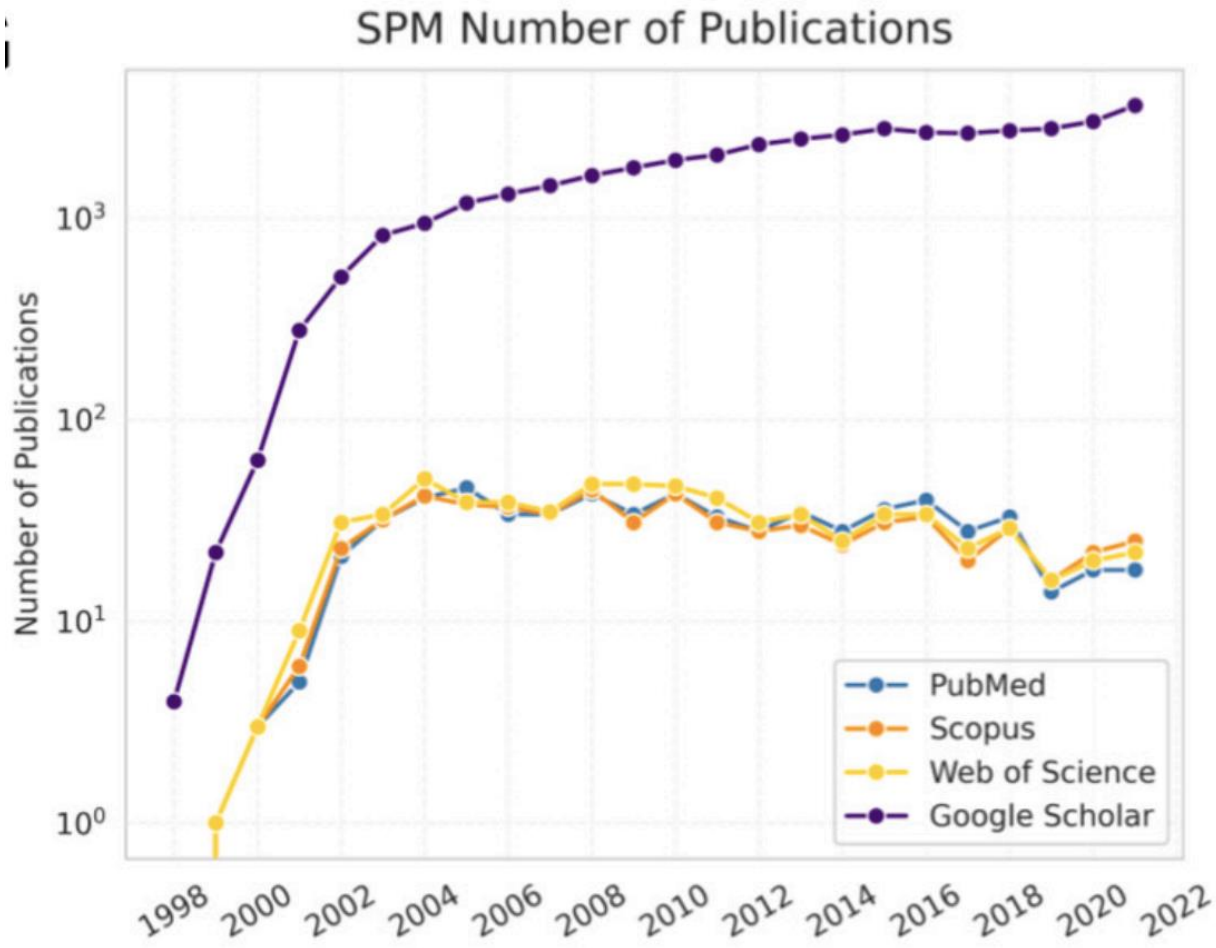
For this study, the report examined the 21 major and minor stable releases s



https://www.sciencedirect.com/science/article/pii/S2405844024024393

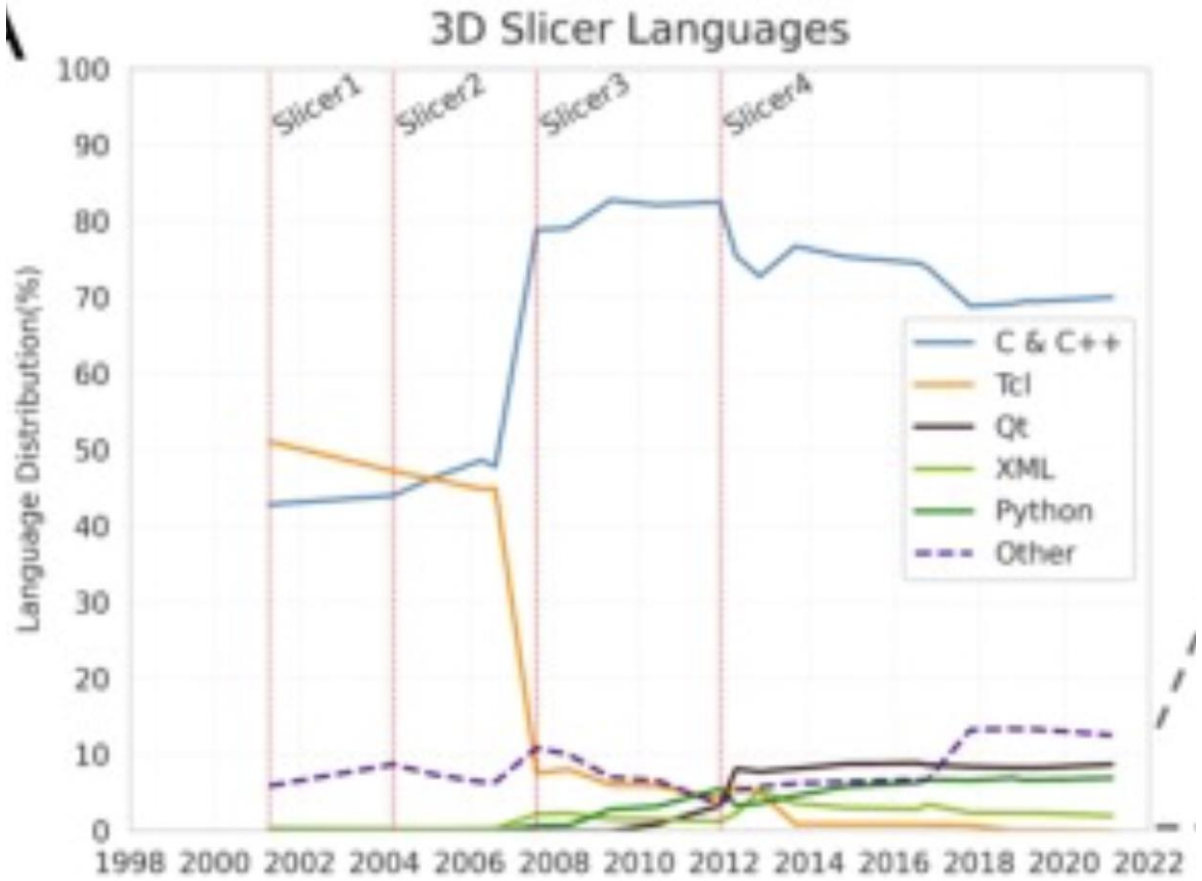
SPM (Statistical Parametric Mapping software)

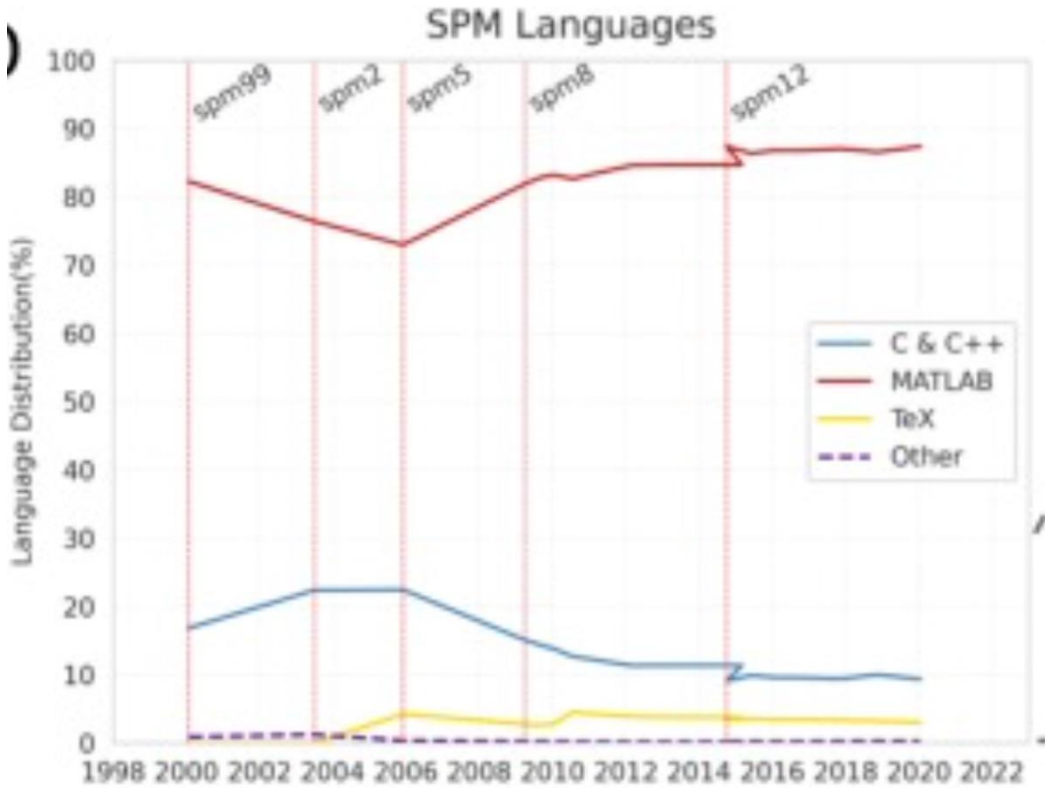
- The (SPM) was born at the MRC Cyclotron Unit, at the Hammersmith Hospital, London
- SPM is available for download for free for macOS, Windows, and Linux operating systems
- 18 major and minor stable releases from 2000 to 2020



https://www.sciencedirect.com/science/article/pii/S2405844024024393

Source Code Evolution





https://www.sciencedirect.com/science/article/pii/S2405844024024393

Thanks for your attention!

Any Question?

Email me on : junaidakram@cuilahore.edu.pk